

InovaGAB

Plataforma de Inovação Corporativa — Grupo Águia Branca

Documentação Técnica — Challenge FIAP · Sprint 2

Integrante	RM
[PREENCHER NOME COMPLETO]	[PREENCHER RM]
[PREENCHER NOME COMPLETO]	[PREENCHER RM]
[PREENCHER NOME COMPLETO]	[PREENCHER RM]
[PREENCHER NOME COMPLETO]	[PREENCHER RM]
[PREENCHER NOME COMPLETO]	[PREENCHER RM]

■ Substitua os campos acima pelos nomes completos e RM de cada integrante antes de enviar a entrega.

Turma: [PREENCHER] · Data de entrega: 21/09/2026
Repositório: github.com/joaovm88/InovacaoGAB-Challenge-FIAP

1. Introdução

Na **Sprint 1** do Challenge, o grupo desenvolveu o aplicativo mobile (Android/Kotlin) da plataforma InovaGAB, com dados simulados (mocks) localmente. Nesta **Sprint 2**, o mock foi substituído por um backend real, transformando o protótipo em uma plataforma funcional de gestão do conhecimento e da inovação para o Grupo Águia Branca.

O sistema permite que **operadores** registrem ideias e problemas do dia a dia, que **gestores** priorizem e aprovem as melhores propostas, transformando-as em projetos, e que a **liderança** defina as orientações estratégicas da empresa e acompanhe os resultados consolidados em um dashboard executivo. Uma camada de **Inteligência Artificial** (Google Gemini) analisa automaticamente cada ideia registrada e sugere uma pontuação de viabilidade, auxiliando os gestores na priorização.

Entregáveis

- Backend RESTful completo (Java 17 + Spring Boot 3 + MongoDB), com autenticação JWT e controle de acesso por perfil.
- Aplicativo Android (Kotlin + Jetpack Compose) integrado ao backend real via Retrofit, sem uso de mocks.
- Integração com IA generativa (Google Gemini) para pontuação automática das ideias de inovação.
- Documentação técnica (este documento): arquitetura, endpoints e modelo de IA utilizado.

2. Arquitetura da Solução

A solução segue uma arquitetura em camadas clássica de uma API REST Spring Boot, consumida por um cliente mobile nativo. A comunicação entre o app e o backend é feita via HTTPS/JSON, autenticada por token JWT enviado no cabeçalho `Authorization: Bearer <token>`. O backend, por sua vez, consulta a API externa do Google Gemini sempre que uma nova ideia é cadastrada, para obter a pontuação de IA.

InovaGAB — Arquitetura da Solução (Sprint 2)

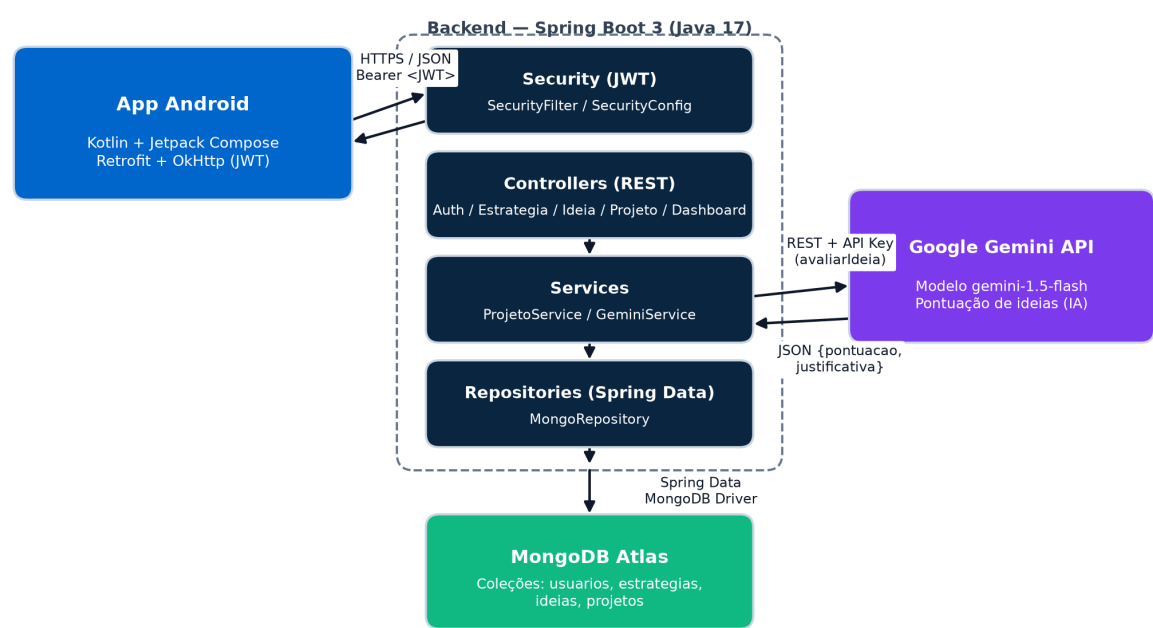


Diagrama de arquitetura: fluxo de dados entre app, backend, banco de dados e IA externa.

Organização em camadas (backend)

- **controller** — expõe os endpoints REST (Auth, Estratégia, Ideia, Projeto, Dashboard).
- **security** — filtro JWT, geração/validação de token e regras de autorização por role.
- **service** — regras de negócio (ex.: validação de investimento) e integração com a IA (GeminiService).
- **repository** — interfaces Spring Data MongoDB para persistência.
- **model** — documentos MongoDB (entidades) mapeados por coleção.
- **dto** — objetos de transferência de dados usados nas requisições/respostas.

3. Stack Tecnológica

Camada	Tecnologias
Backend	Java 17, Spring Boot 3.3.4 (Web, Security, Data MongoDB, Validation), java-jwt (Auth0), Lombok, JUnit 5 + Mockito
Banco de Dados	MongoDB Atlas (NoSQL, documentos)
Autenticação	JWT (JSON Web Token) + BCrypt para hash de senha
Inteligência Artificial	Google Gemini API (modelo gemini-1.5-flash)

Camada	Tecnologias
App Mobile	Kotlin, Jetpack Compose, Navigation Compose, Retrofit2 + OkHttp3 (cliente HTTP), Gson, Kotlin Coroutines
Build/Infra	Maven (backend), Gradle Kotlin DSL (app)

4. Modelo de Dados (Coleções MongoDB)

usuarios (Usuario)

- id, nome, email (único), senha (hash BCrypt), nivelAcesso (OPERADOR | GESTOR | LIDER)

estrategias (Estrategia)

- id, categoria, campanha, descricao, dataCriacao, ativa (boolean)

ideias (IdeaInovacao)

- id, titulo, descricao, setor, autorId, estrategiaId
- status (PENDENTE | APROVADA | ARQUIVADA), feedbackGestor
- pontuacao (0–100), justificativa — preenchidos automaticamente pela IA no cadastro
- dataSubmissao

projetos (Projeto)

- id, nome, descricao, etapa (fase), divisao, ideiaOrigemId, estrategiaId
- investimento, retornoFinanceiro, reducaoCustos, roi (métricas financeiras)
- co2EvitadoToneladas, aguaPoupadaLitros (métricas ESG)
- dataInicio, prazoFinal, finalizado (boolean)

5. Autenticação e Perfis de Acesso

A autenticação é feita via **JWT**: no login, o backend valida e-mail/senha e devolve um token assinado (HMAC256) contendo o id do usuário, o e-mail (subject) e o perfil de acesso (role) como claims. Esse token deve ser enviado em todas as chamadas subsequentes no cabeçalho `Authorization: Bearer <token>`. O `SecurityFilter` valida o token a cada requisição e o Spring Security aplica as regras de autorização por rota, de acordo com a role do usuário.

Perfil	Permissões
OPERADOR	Cadastra, consulta, edita e exclui as próprias ideias de inovação.
GESTOR	Consulta e avalia (aprova/arquiva) as ideias pendentes; gerencia (CRUD) os projetos/iniciativas.
LIDER	Gerencia (CRUD) as orientações estratégicas; consulta projetos; acessa o dashboard executivo.

6. Especificação dos Endpoints

Base URL local: `http://localhost:8080`

6.1 Autenticação — `/api/auth` (públicos)

Método	Rota	Acesso	Payload	Resposta
POST	<code>/api/auth/registro</code>	Público	<code>{"nome","email","senha","nivelAcesso":"OPERADOR GESTOR LIDER"}</code>	201 Criado / 409 e-mail já existe
POST	<code>/api/auth/login</code>	Público	<code>{"email","senha"}</code>	200 <code>{"token","tipo":"Bearer","nivelAcesso"}</code>

6.2 Estratégias — `/api/estrategias` (LIDER gerencia, demais consultam)

Método	Rota	Acesso	Payload	Resposta
POST	<code>/api/estrategias</code>	LIDER	<code>{"categoria","campanha","descricao"}</code>	201 Criada
GET	<code>/api/estrategias</code>	Autenticado	—	200 Lista de estratégias ativas
GET	<code>/api/estrategias/todas</code>	LIDER	—	200 Histórico completo
PUT	<code>/api/estrategias/{id}</code>	LIDER	<code>{"categoria","campanha","descricao"}</code>	200 Atualizada / 404
PATCH	<code>/api/estrategias/{id}/status</code>	LIDER	query: <code>ativa=true false</code>	200 Atualizada / 404
DELETE	<code>/api/estrategias/{id}</code>	LIDER	—	204 Removida / 404

6.3 Ideias de Inovação — `/api/ideias`

Método	Rota	Acesso	Payload	Resposta
POST	<code>/api/ideias</code>	OPERADOR, GESTOR, LIDER	<code>{"titulo","descricao","setor","autorId","estrategiaId"}</code>	201 Criada (com pontuação e justificativa preenchidas pela IA)
GET	<code>/api/ideias/autor/{autorId}</code>	Autenticado	—	200 Ideias do autor
GET	<code>/api/ideias/pendentes</code>	Autenticado	—	200 Ideias com status PENDENTE
PUT	<code>/api/ideias/{id}</code>	Autor, GESTOR ou LIDER	<code>{"titulo","descricao","setor"}</code>	200 Atualizada / 403 / 404
DELETE	<code>/api/ideias/{id}</code>	Autor, GESTOR ou LIDER	—	204 Removida / 403 / 404
PATCH	<code>/api/ideias/{id}/avaliacao</code>	GESTOR, LIDER	query: <code>status=APROVADA ARQUIVADA&parecer;=...</code>	200 Avaliada / 404

6.4 Projetos — /api/projetos (GESTOR gerencia, LIDER consulta)

Método	Rota	Acesso	Payload	Resposta
POST	/api/projetos	GESTOR	{"nome","descricao","etapa","divisao","estrategiaId","investimento",...}	201 Criado / 400 investimento inválido
GET	/api/projetos	GESTOR, LIDER	—	200 Lista de projetos
GET	/api/projetos/{id}	GESTOR, LIDER	—	200 Projeto / 404
PUT	/api/projetos/{id}	GESTOR	{"nome","descricao","divisao","investimento",...}	200 Atualizado / 404
PUT	/api/projetos/{id}/progresso	GESTOR	query: fase=...&retornoFinanceiro;=...&finalizado;=...	200 Atualizado / 404
DELETE	/api/projetos/{id}	GESTOR	—	204 Removido / 404

6.5 Dashboard Executivo — /api/dashboard (exclusivo LIDER)

Método	Rota	Acesso	Payload	Resposta
GET	/api/dashboard/resultados	LIDER	—	200 Resumo: investimentoTotal, retornoTotal, lucroLiquido, roiPercentual, reducaoCustosTotal, co2EvitadoToneladas, aguaPoupadaLitros, totalProjetos

7. Integração com Inteligência Artificial (Diferencial)

Funcionalidade escolhida: Integração com IA para **pontuação e priorização automática** das iniciativas/ideias de inovação enviadas pelos colaboradores, auxiliando os gestores na seleção dos futuros projetos.

Modelo utilizado: gemini-1.5-flash, da **Google Gemini API** (API gratuita/free tier), acessado via requisição REST autenticada por API key.

Como funciona

- Toda vez que uma ideia é cadastrada (POST /api/ideias), o GeminiService monta um prompt contextual com o título, o setor, a descrição do problema/solução e a estratégia corporativa vigente vinculada à ideia.
- Esse prompt é enviado à Gemini API pedindo uma resposta estritamente em JSON, no formato {"pontuacao": 0-100, "justificativa": "texto"}.
- A resposta é interpretada e os campos pontuacaoIa e justificativaIa são gravados junto com a ideia, ficando visíveis para o gestor no momento da priorização (inclusive no app).
- **Fallback seguro:** se a chave GEMINI_API_KEY não estiver configurada, ou a chamada externa falhar/expirar, o serviço aplica uma pontuação padrão (50 ou 60, conforme o caso) com uma justificativa informativa — o cadastro da ideia nunca é bloqueado por indisponibilidade da IA.

Prompt utilizado (resumo)

Você é um consultor sênior de inovação e ESG do Grupo Águia Branca. Avalie a proposta considerando: título, setor, descrição do problema/solução e a estratégia corporativa vigente. Responda apenas com um JSON contendo pontuacao (0 a 100) e justificativa (texto curto).

8. Integração do Aplicativo com o Backend

O aplicativo Android consome a API REST via **Retrofit + OkHttp**. Um interceptor HTTP injeta automaticamente o cabeçalho Authorization: Bearer <token> em toda requisição, a partir do token salvo localmente (SharedPreferences) no momento do login. Como o backend não retorna o id/e-mail do usuário no corpo da resposta de login, esses dados são extraídos diretamente do payload do próprio JWT no dispositivo.

Tela do App	Endpoints consumidos
Login / Cadastro	POST /api/auth/login, POST /api/auth/registro
Mural (Estratégias)	GET /api/estrategias, POST /api/estrategias, DELETE /api/estrategias/{id} (Líder)
Ideias	POST /api/ideias, GET /api/ideias/autor/{id}, GET /api/ideias/pendentes, PUT/DELETE /api/ideias/{id}, PATCH /api/ideias/{id}/avaliacao
Feedbacks	GET /api/ideias/autor/{id} (ideias já avaliadas do próprio operador)
Projetos	GET /api/projetos, POST /api/projetos, PUT /api/projetos/{id}/progresso (Gestor)

Tela do App	Endpoints consumidos
Dashboard	GET /api/dashboard/resultados, GET /api/projetos (Líder)

9. Como Executar o Projeto

Backend (IntelliJ IDEA ou terminal)

- Abrir a pasta inovagab/inovagab como projeto Maven.
- Configurar as variáveis de ambiente MONGODB_URI (obrigatória) e GEMINI_API_KEY (opcional, ativa a IA de verdade).
- Rodar a classe InovagabApplication, ou `./mvnw spring-boot:run`. A API sobe em `http://localhost:8080`.

App Android (Android Studio)

- Abrir a raiz do repositório no Android Studio e sincronizar o Gradle.
- Rodar em um emulador — o app já aponta para `http://10.0.2.2:8080`, que é como o emulador enxerga o backend rodando na própria máquina.
- Para testar em um aparelho físico, trocar a URL em `ApiClient.kt` pelo IP da máquina na rede local.

Testes automatizados (backend)

```
cd inovagab/inovagab && ./mvnw test
```

Cobre autenticação/geração de token, regra de negócio de investimento de projetos e o fallback do serviço de IA quando a chave não está configurada.

10. Considerações Finais e Evoluções Futuras

- Autenticação/autorização completas com JWT e 3 perfis de acesso, com regras aplicadas tanto no backend quanto no app.
- CRUD completo para Estratégias, Ideias e Projetos, com checagem de propriedade nas ideias (autor, gestor ou líder).
- Dashboard executivo com dados agregados reais (investimento, retorno, ROI, redução de custos, indicadores ESG).
- Diferencial de IA generativa (Google Gemini) plenamente funcional, com fallback seguro.

Possíveis evoluções futuras: vínculo automático de projeto a partir de uma ideia aprovada, notificações push para o operador quando sua ideia é avaliada, e um chat com IA para apoiar o gestor na priorização (segunda opção de IA sugerida no enunciado).